דו״ח מעבדה 2

מימוש Behavioral ו-Structural של Adder ברוחב 8-bit

מגישים :
מוחמד משעל

תוכן עניינים

תוכן עניינים 2

1. מבוא ומטרת המעבדה 3
2. תהליך התכנון 3
   2.1 מימוש Behavioral 3
   2.2 מימוש Structural (Ripple-Carry Adder) 3
3. קוד המקור (Verilog) 3
   3.1 מימוש Behavioral 3
   3.2 מימוש Structural (Ripple-Carry) 4

4. Testbench משותף 5
5. תוצאות סימולציה 6
   5.1 מקרי הבדיקה (Test Cases) 6
   5.2 פלט הקונסולה (Simulation Log) 6
   5.3 Waveform 6

6. מימוש פיזי (Physical Implementation) 7
   6.1 קוד ה-Top Level (chip_top) 7
   6.2 קובץ אילוצים (Constraints- XDC) 7
   6.3 סכמות Lint (Schematics) 8
   6.4 דו״חות Synthesis ו-Implementation 8
   6.5 דו״ח שימוש במשאבים (Resource Utilization) 9
   6.6 בדיקה על לוח Nexys 9

7. השוואה בין המימושים וסיכום 10
   7.1 טבלת השוואה 10
   7.2 סיכום כללי 10

1. מבוא ומטרת המעבדה

במעבדה הזו תכננו ובדקנו שני מימושים שונים ל Adder-ברוחב 8 ביטים עם כניסת carry-in ויציאת carry-out: מימוש אחד ברמת Behavioral, שבו פשוט השתמשנו באופרטור החיבור של Verilog, ומימוש שני ברמת Structural, שבו בנינו את הלוגיקה בעצמנו ברמת הביט, בשיטת Ripple-Carry. המטרה הייתה להשוות בין שתי הגישות מבחינת קריאות הקוד, קלות הכתיבה, ובעיקר מבחינת השימוש במשאבי החומרה בפועל

2. תהליך התכנון

2.1 מימוש Behavioral

במימוש הזה לא כתבתי לוגיקה ברמת הביט בכלל, אלא פשוט חיברתי את a, b ו-cin באמצעות אופרטור החיבור הרגיל. כדי לא לאבד את ה-carry-out חישבתי את התוצאה לתוך משתנה ברוחב 9 ביטים, ואז לקחתי את 8 הביטים הנמוכים כ-sum ואת הביט התשיעי (העליון) כ-cout.

2.2 מימוש Structural (Ripple-Carry Adder)

במימוש הזה בניתי full adder בודד ברמת ביט אחד, עם המשוואות הרגילות של sum ו-carry. אחר כך שרשרתי 8 עותקים של אותו full adder, כך שה-carry-out של כל ביט מוזן כ-carry-in לביט הבא. ה-carry-in הראשון של השרשרת הוא ה-cin החיצוני, וה-carry-out האחרון הוא ה-cout החיצוני.

3. קוד המקור (Verilog)

3.1 מימוש Behavioral

```
1  `timescale 1ns / 1ps
2
3  module adder8_behavioral (
4      input  wire [7:0] a,
5      input  wire [7:0] b,
6      input  wire       cin,
7      output reg  [7:0] sum,
8      output reg        cout
9  );
10
11     always @(*) begin
12         {cout, sum} = {1'b0, a} + {1'b0, b} + cin;
13     end
14
15 endmodule
```

```
`timescale 1ns / 1ps

module adder8_structural (
    input  wire [7:0] a,
    input  wire [7:0] b,
    input  wire       cin,
    output wire [7:0] sum,
    output wire       cout
);

    wire [8:0] c;

    assign c[0] = cin;

    // Bit 0
    assign sum[0] = a[0] ^ b[0] ^ c[0];
    assign c[1] = (c[0] & (a[0] ^ b[0])) | (a[0] & b[0]);

    // Bit 1
    assign sum[1] = a[1] ^ b[1] ^ c[1];
    assign c[2] = (c[1] & (a[1] ^ b[1])) | (a[1] & b[1]);

    // Bit 2
    assign sum[2] = a[2] ^ b[2] ^ c[2];
    assign c[3] = (c[2] & (a[2] ^ b[2])) | (a[2] & b[2]);

    // Bit 3
    assign sum[3] = a[3] ^ b[3] ^ c[3];
    assign c[4] = (c[3] & (a[3] ^ b[3])) | (a[3] & b[3]);


    // Bit 4
    assign sum[4] = a[4] ^ b[4] ^ c[4];
    assign c[5] = (c[4] & (a[4] ^ b[4])) | (a[4] & b[4]);

    // Bit 5
    assign sum[5] = a[5] ^ b[5] ^ c[5];
    assign c[6] = (c[5] & (a[5] ^ b[5])) | (a[5] & b[5]);

    // Bit 6
    assign sum[6] = a[6] ^ b[6] ^ c[6];
    assign c[7] = (c[6] & (a[6] ^ b[6])) | (a[6] & b[6]);

    // Bit 7
    assign sum[7] = a[7] ^ b[7] ^ c[7];
    assign c[8] = (c[7] & (a[7] ^ b[7])) | (a[7] & b[7]);

    assign cout = c[8];

endmodule
```

4. Testbench משותף

```
`timescale 1ns / 1ps

module tb_adder8;

    reg [7:0] a, b;
    reg cin;

    wire [7:0] sum_beh, sum_str;
    wire cout_beh, cout_str;

    adder8_behavioral dut_beh (
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum_beh),
        .cout(cout_beh)
    );

    adder8_structural dut_str (
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum_str),
        .cout(cout_str)
    );

    initial begin
        // Test 1: No carry-out
        $display("Time: 0 ns");
        a = 8'd3; b = 8'd4; cin = 1'b0; #10;

        // Test 2: With carry-out
        $display("Time: 10 ns");
        a = 8'd200; b = 8'd100; cin = 1'b0; #10;

        // Test 3: Maximum sum without carry-out
        $display("Time: 20 ns");
        a = 8'd255; b = 8'd0; cin = 1'b0; #10;

        // Test 4: Maximum total result
        $display("Time: 30 ns");
        a = 8'd255; b = 8'd255; cin = 1'b1; #10;

        // Test 5: Carry-in propagation
        $display("Time: 40 ns");
        a = 8'd255; b = 8'd0; cin = 1'b1; #10;

        $finish;
    end

    initial begin
         $monitor("a=%b, b=%b, cin=%b, sum_beh=%b, cout_beh=%b, sum_str=%b, cout_str=%b", a, b, cin, sum_beh, cout_beh, sum_str, cout_str);
    end

endmodule
```

5. תוצאות סימולציה

5.1 מקרי הבדיקה (Test Cases)

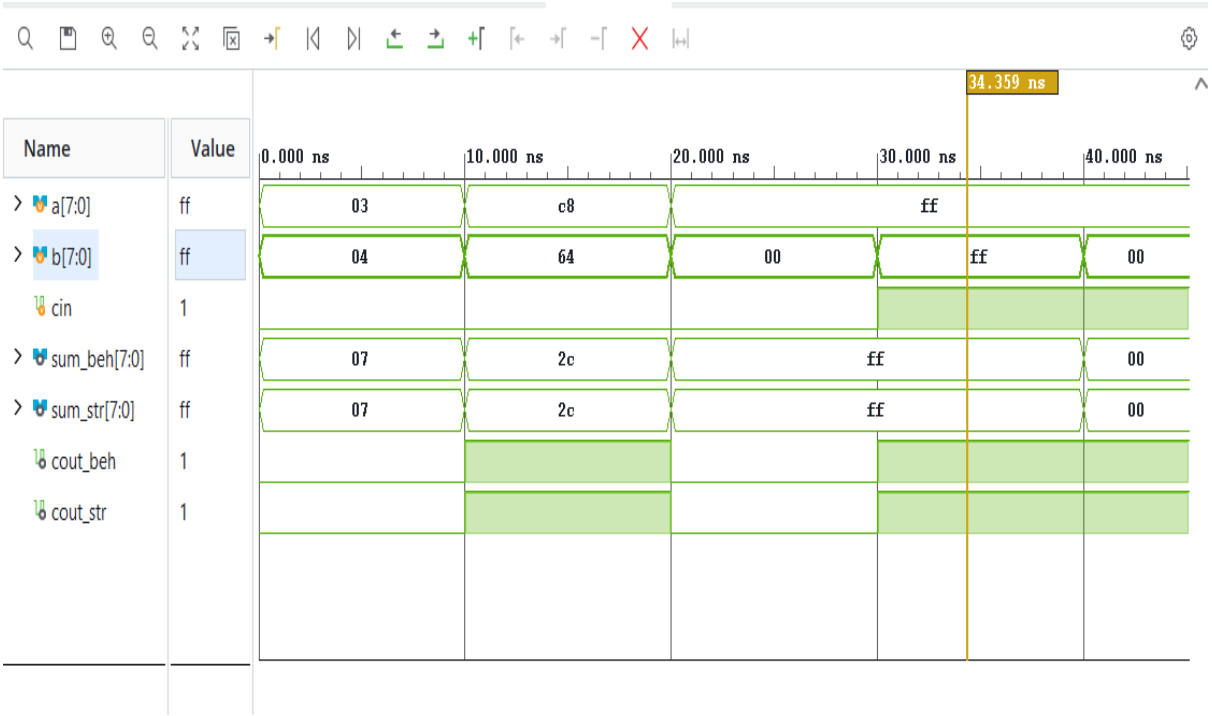
טבלה מסכמת של 5 מקרי הבדיקה שהוגדרו:

#	תיאור המקרה	cin	b	a	צפוי sum	צפוי cout
1	ללא carry-out	0	4	3	7	0
2	עם carry-out	0	100	200	44	1
3	ללא carry-out, ערך מרבי	0	0	255	255	0
4	מרבי overflow	1	255	255	255	1
5	מקרה נוסף (משלים ל-5)	1	0	255	0	1

5.2 פלט הקונסולה (Simulation Log)



Waveform 5.3



6. מימוש פיזי (Physical Implementation)

למימוש הפיזי על לוח ה-Nexys בחרתי להשתמש במימוש Behavioral

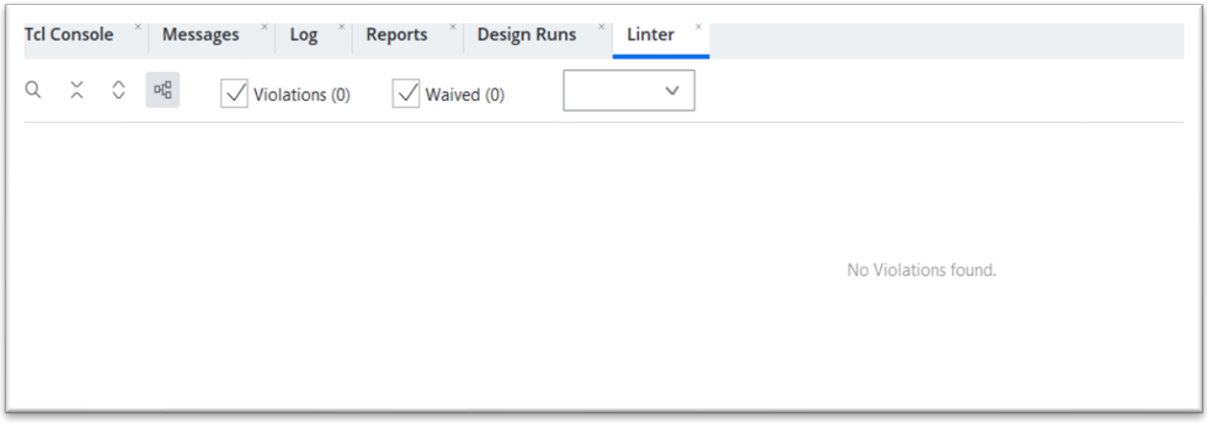
6.1 קוד ה-Top Level (chip_top)

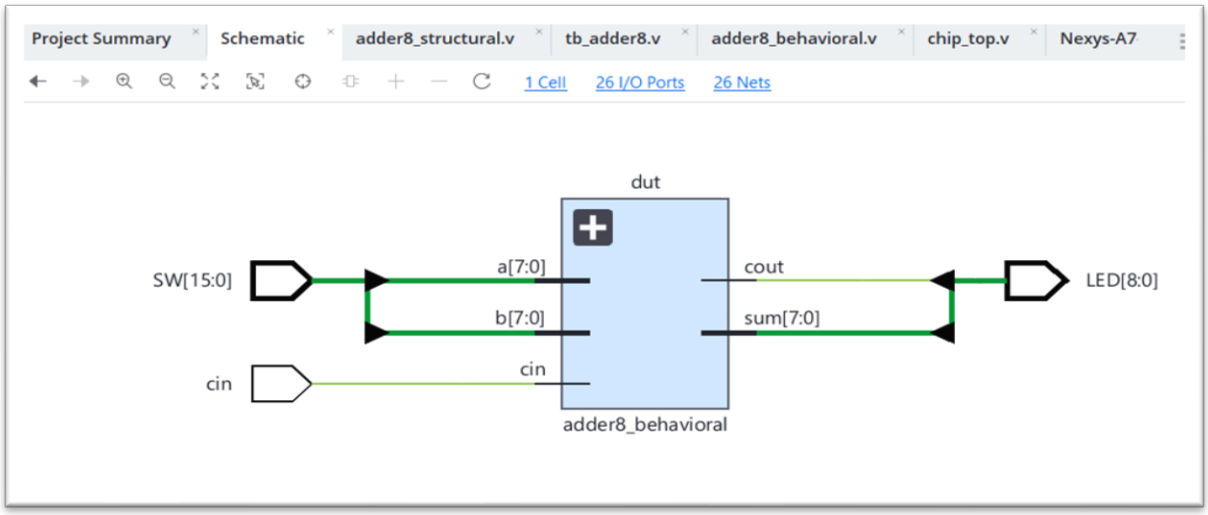
```
1  `timescale 1ns / 1ps
2
3  module chip_top (
4      input  wire [15:0] SW,
5      input  wire        cin,
6      output wire [8:0]  LED
7  );
8
9      adder8_behavioral dut (
10         .a(SW[7:0]),
11         .b(SW[15:8]),
12         .cin(cin),
13         .sum(LED[7:0]),
14         .cout(LED[8])
15     );
16
17 endmodule
```

6.2 קובץ אילוצים (Constraints- XDC)

```
13 set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
14 set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
15 set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
16 set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
17 set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
18 set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { SW[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
19 set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { SW[6] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
20 set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
21 set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }]; #IO_L24N_T3_34 Sch=sw[8]
22 set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }]; #IO_25_34 Sch=sw[9]
23 set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { SW[10] }]; #IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
24 set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { SW[11] }]; #IO_L23P_T3_A03_D19_14 Sch=sw[11]
25 set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }]; #IO_L24P_T3_35 Sch=sw[12]
26 set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }]; #IO_L20P_T3_A08_D24_14 Sch=sw[13]
27 set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
28 set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }]; #IO_L21P_T3_DQS_14 Sch=sw[15]
29
30 ## LEDs
31 set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { LED[0] }]; #IO_L18P_T2_A24_15 Sch=led[0]
32 set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { LED[1] }]; #IO_L24P_T3_RS1_15 Sch=led[1]
33 set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { LED[2] }]; #IO_L17N_T2_A25_15 Sch=led[2]
34 set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { LED[3] }]; #IO_L8P_T1_D11_14 Sch=led[3]
35 set_property -dict { PACKAGE_PIN R18   IOSTANDARD LVCMOS33 } [get_ports { LED[4] }]; #IO_L7P_T1_D09_14 Sch=led[4]
36 set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports { LED[5] }]; #IO_L18N_T2_A11_D27_14 Sch=led[5]
37 set_property -dict { PACKAGE_PIN U17   IOSTANDARD LVCMOS33 } [get_ports { LED[6] }]; #IO_L17P_T2_A14_D30_14 Sch=led[6]
38 set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports { LED[7] }]; #IO_L18P_T2_A12_D28_14 Sch=led[7]
39 set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { LED[8] }]; #IO_L16N_T2_A15_D31_14 Sch=led[8]
```

6.3 סכמות (Schematics) Lint

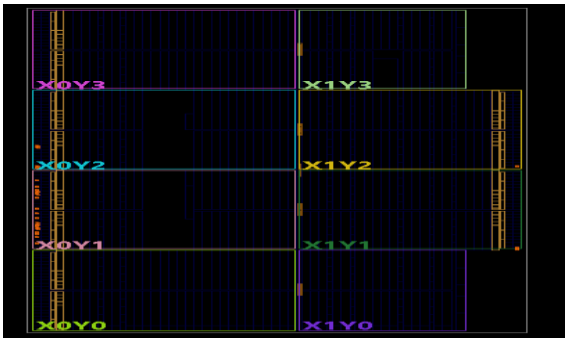


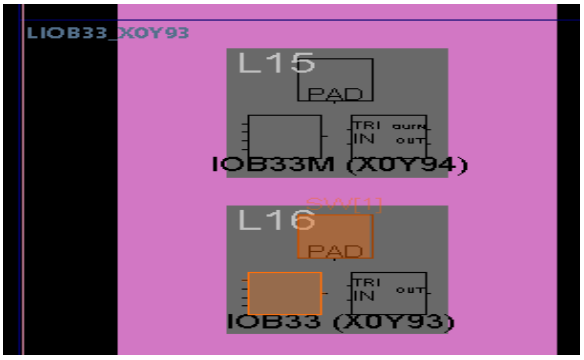


6.4 דו״חות Synthesis ו-Implementation

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM
✓ synth_1	constrs_1	synth_design Complete!												8	0	0	0
✓ impl_1	constrs_1	route_design Complete!	NA	NA	NA	NA		NA	18.655	0				8	0	0	0

DSP	Start	Elapsed	Run Strategy	Report Strategy	Part	Description
0	9/27/26, 1:34 PM	00:00:58	Vivado Synthesis Defaults (Vivado Synthesis 2026)	Vivado Synthesis Default Reports (Vivado Synthesis 2026)	xc7a100tcsg324-1	Vivado Synthesis Defaults
0	9/27/26, 1:49 PM	00:01:18	Vivado Implementation Defaults (Vivado Implementation 2026)	Vivado Implementation Default Reports (Vivado Implementation 2026)	xc7a100tcsg324-1	Default settings for Implementation.





6.5 דו״ח שימוש במשאבים (Resource Utilization)

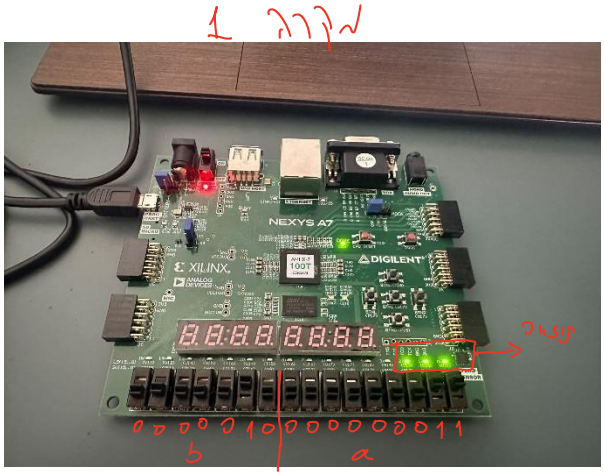
Utilization | Post-Synthesis | **Post-Implementation**

Graph | **Table**

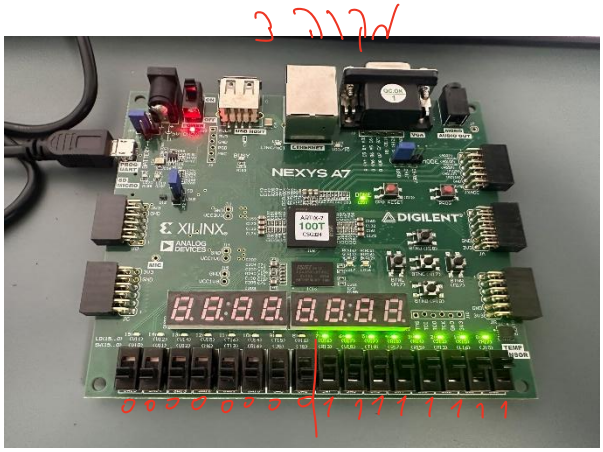
Resource	Utilization	Available	Utilization %
LUT	8	63400	0.01
IO	26	210	12.38

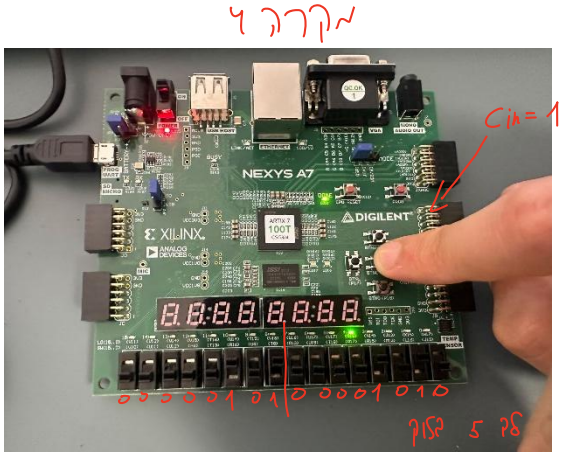
מהטבלה ניתן לראות שהמעגל צרך 8 LUTs מתוך 63,400 הזמינים (0.01%) ו-26 פינים דיגיטליים (IO) מתוך 210 הזמינים (12.38%). מספר ה-LUTs הנמוך מאוד תואם את הציפיות, מכיוון ש-Adder הוא מעגל קומבינטורי פשוט יחסית, ו-Vivado ככל הנראה מיפה את החיבור למעגלי carry-chain ייעודיים בשבב (CARRY4) במקום ל-LUTs רגילים, מה שמסביר את הצריכה הזעירה. מספר ה-IO (26) תואם בדיוק את מספר הפורטים החיצוניים של המעגל: 17 כניסות (8 ביטים של a, 8 ביטים של b, וביט אחד של cin) ו-9 יציאות (8 ביטים של sum וביט אחד של cout). לכן ניתן לומר שהשימוש במשאבים תואם באופן מלא לציפיות התיאורטיות

6.6 בדיקה על לוח Nexys









7. השוואה בין המימושים וסיכום

7.1 טבלת השוואה

קריטריון	Structural (Ripple-Carry)	Behavioral
קריאות/תחזוקתיות הקוד	הרבה יותר ארוך	קצר וקריא מאוד
שליטה על ה-hardware בפועל	שליטה מלאה	אין שליטה מלאה
שימוש במשאבים (LUTs/FFs)	LUT=8, FF=0	LUT=8, FF=0
זמן עיכוב (delay) משוער	inf	inf
קלות דיבוג	קשה יותר	קל מאוד

7.2 סיכום כללי

במעבדה זו תכננו, מימשנו והשווינו שני מימושים שונים ל-Adder ברוחב 8 ביטים: מימוש Behavioral, שבו הסתפקנו באופרטור החיבור המובנה של Verilog, ומימוש Structural, שבו בנינו בעצמנו שרשרת של 8 full-adders ברמת הביט, בשיטת Ripple-Carry. מבחינת כתיבת הקוד, מצאנו שהמימוש ה-Behavioral קצר וקריא בהרבה, בעוד שהמימוש ה-Structural דורש הבנה מעמיקה יותר של המבנה הפנימי ומועד יותר לטעויות, אך מעניק שליטה מפורשת על כל ביט וכל שרשרת carry.

באופן מפתיע, בבדיקת דו"חות ה-Synthesis וה-Utilization מצאנו ששני המימושים הובילו לאותו ניצול משאבים בדיוק (8 LUTs, 0 Flip-Flops), מה שמלמד שכלי הסינתזה של Vivado מזהה את תבנית ה-adder גם כשהיא נכתבת בשתי צורות שונות, וממפה את שתיהן לאותה שרשרת carry ייעודית בשבב (CARRY4). מכיוון שלא הגדרנו אילוצי שעון בפרויקט, לא היה בידינו למדוד ולהשוות זמני עיכוב בפועל, אך מבחינה תיאורטית ציפינו שבמימוש ה-Structural זמן ההתפשטות (propagation delay) יגדל ככל שרוחב הביטים גדל, בשל ההמתנה לשרשרת ה-carry.

בבדיקת הסימולציה ובבדיקה הפיזית על לוח ה-Nexys קיבלנו התאמה מלאה בין שני המימושים לבין התוצאות הצפויות, בכל חמשת מקרי הבדיקה. בפרט, הבדיקה הפיזית איפשרה לנו להדגים בצורה מוחשית כיצד ה-carry-in משפיע רק על הביט הנמוך ביותר של הסכום כאשר אין המשך שרשרת, כתוצאה ישירה ממנגנון ה-Ripple-Carry. לסיכום, למדנו שהגישה ה-Behavioral מתאימה יותר לפיתוח מהיר ולבדיקת פונקציונליות, בעוד שהגישה ה-Structural מתאימה יותר כאשר יש צורך בשליטה מפורשת ומדויקת על מבנה החומרה.